

React Custom Hooks

Eigene Hooks erstellen und wiederverwenden

1. Was ist ein Custom Hook?

Ein Custom Hook ist eine **normale JavaScript-Funktion**, die mit **use** beginnt und andere React Hooks (useState, useEffect, useRef, ...) verwenden darf. Mit Custom Hooks kann man **wiederverwendbare Logik** aus Komponenten auslagern – statt dieselbe useState/useEffect-Kombination in mehreren Komponenten zu wiederholen, schreibt man sie einmal und verwendet sie überall.

Merksatz: Custom Hook = Logik auslagern + wiederverwenden → Komponente bleibt klein und lesbar.

2. Die wichtigsten Regeln

Regel	Erklärung
Name beginnt mit use	React erkennt es als Hook und prüft die Hook-Regeln
Nur in Funktionskomponenten	Nicht in Klassen oder normalen Funktionen aufrufen
Nicht in Bedingungen	Hooks immer auf der obersten Ebene aufrufen
Eigene State-Instanz	Jede Komponente bekommt ihren eigenen unabhängigen State

3. Beispiel 1 – useFetch (Daten von API laden)

Das häufigste Szenario: Mehrere Komponenten laden Daten von einer API. Ohne Custom Hook schreibt man useState + useEffect dreimal. Mit **useFetch** einmal.

Der Custom Hook – useFetch.js

```
import { useState, useEffect } from 'react';

function useFetch(url) {
  const [daten, setDaten] = useState(null);
  const [laden, setLaden] = useState(true);
  const [fehler, setFehler] = useState(null);

  useEffect(() => {
    fetch(url)
      .then(res => res.json())
      .then(data => {
        setDaten(data);
        setLaden(false);
      })
      .catch(err => {
        setFehler(err.message);
        setLaden(false);
      });
  });
}
```

```

    });
  }, [url]);

  return { daten, laden, fehler };
}

export default useFetch;

```

Verwendung in einer Komponente

```

import useFetch from './useFetch';

function Benutzerliste() {
  const { daten, laden, fehler } = useFetch('https://api.example.com/users');

  if (laden) return <p>Lädt...</p>;
  if (fehler) return <p>Fehler: {fehler}</p>;

  return (
    <ul>
      {daten.map(user => (
        <li key={user.id}>{user.name}</li>
      ))}
    </ul>
  );
}

```

Die gesamte Lade-Logik (fetch, Fehlerzustand, Ladezustand) steckt in useFetch. Die Komponente selbst bleibt kurz und übersichtlich – und jede andere Komponente kann denselben Hook mit einer anderen URL verwenden.

4. Beispiel 2 – useLocalStorage (Wert im Browser speichern)

useLocalStorage verhält sich wie **useState**, speichert den Wert aber automatisch im **localStorage** des Browsers – der Wert bleibt also auch nach einem Seiten-Reload erhalten.

Der Custom Hook – useLocalStorage.js

```

import { useState } from 'react';

function useLocalStorage(schlüssel, initialWert) {
  const [wert, setWert] = useState(() => {
    // Beim ersten Aufruf: gespeicherten Wert lesen
    const gespeichert = localStorage.getItem(schlüssel);
    return gespeichert ? JSON.parse(gespeichert) : initialWert;
  });

  const speichern = (neuerWert) => {
    setWert(neuerWert);
    localStorage.setItem(schlüssel, JSON.stringify(neuerWert));
  };

  return [wert, speichern];
}

```

```
export default useLocalStorage;
```

Verwendung in einer Komponente

```
import useLocalStorage from './useLocalStorage';

function Einstellungen() {
  const [thema, setThema] = useLocalStorage('thema', 'hell');

  return (
    <div>
      <p>Aktuelles Thema: {thema}</p>
      <button onClick={() => setThema(thema === 'hell' ? 'dunkel' : 'hell')}>
        Thema wechseln
      </button>
    </div>
  );
}
```

Das API ist identisch zu useState: Array-Destructuring mit [wert, setterFunktion]. Der Unterschied ist unsichtbar – er steckt komplett im Hook.

5. Beispiel 3 – useDebounce (Eingabe verzögern)

Bei Suchfeldern möchte man nicht bei jedem Tastendruck eine API anfragen. **useDebounce** verzögert die Weitergabe eines Wertes – erst wenn der Nutzer aufgehört hat zu tippen, wird der Wert aktualisiert.

Der Custom Hook – useDebounce.js

```
import { useState, useEffect } from 'react';

function useDebounce(wert, verzoegerung = 500) {
  const [debouncedWert, setDebouncedWert] = useState(wert);

  useEffect(() => {
    // Timer starten - läuft ab, wenn der Nutzer aufgehört hat zu tippen
    const timer = setTimeout(() => {
      setDebouncedWert(wert);
    }, verzoegerung);

    // Cleanup: Timer abbrechen, wenn wert sich erneut ändert
    return () => clearTimeout(timer);
  }, [wert, verzoegerung]);

  return debouncedWert;
}

export default useDebounce;
```

Verwendung in einer Suchkomponente

```
import { useState, useEffect } from 'react';
import useDebounce from './useDebounce';

function Suche() {
```

```

const [eingabe, setEingabe] = useState('');
const verzogert = useDebounce(eingabe, 400);

useEffect(() => {
  if (verzogert) {
    console.log('API-Aufruf mit:', verzogert);
    // Hier: fetch(`/api/suche?q=${verzogert}`)
  }
}, [verzogert]);

return (
  <input
    value={eingabe}
    onChange={e => setEingabe(e.target.value)}
    placeholder="Suche..."
  />
);
}

```

Der Cleanup-Return im useEffect ist entscheidend: Jedes Mal wenn der Nutzer tippt, wird der alte Timer abgebrochen und ein neuer gestartet. Nur der letzte Timer läuft durch.

6. Custom Hook vs. normale Funktion

Nicht jede ausgelagerte Funktion ist ein Custom Hook. Der Unterschied:

	Custom Hook	Normale Funktion
Beginnt mit use	Ja (Pflicht)	Nein
Darf Hooks verwenden	Ja	Nein
Gibt React-State zurück	Ja	Nein
Beispiel	useFetch, useDebounce	formatDatum, berechnePreis

7. Zusammenfassung

Custom Hooks sind kein spezielles React-Feature – sie sind **Konvention und guter Stil**. Sobald man dieselbe useState/useEffect-Kombination in mehreren Komponenten schreibt, ist es Zeit für einen Custom Hook. Die drei wichtigsten Vorteile:

Vorteil	Beschreibung
Wiederverwendbarkeit	Einmal schreiben, überall verwenden
Lesbarkeit	Komponenten bleiben kurz und fokussiert
Testbarkeit	Logik lässt sich unabhängig von der UI testen

Faustregel: Wenn du dich beim Kopieren von Hook-Logik erwischst, erstelle einen Custom Hook!